

Mecánicas de "Rango de Filtrado" para Snake

Diagnóstico de arquitectura y propuesta de mecánicas de juego para un estudio experimental de Vibe Coding

Proyecto: **Game-01** (Snake, Python / pygame) · Rama: **Game-01/Snake**

Elaborado por: Asesoría de arquitectura de software y desarrollo de videojuegos · 26 de julio de 2026

0. Resumen ejecutivo

Este informe evalúa la base de código de **Game-01**, una implementación completa y funcional de Snake en Python/pygame, con el propósito de identificar puntos de extensión idóneos para un estudio de **vibe coding** (programación asistida por IA mediante instrucciones en lenguaje natural). Se propone un conjunto de mecánicas de juego articuladas alrededor de un concepto común —el **rango de filtrado**— expresado en tres variantes: filtrado espacial (radio de detección), filtrado de valores (rango numérico de consumo) y filtrado de coordenadas (zonas rectangulares de alteración física). Cada mecánica se acompaña de un análisis de viabilidad pedagógica —qué instrucciones debe dar el participante y qué errores típicos de la IA deberá detectar y corregir— y de un mapa preciso de los archivos y funciones donde debe inyectarse el código nuevo.

1. Diagnóstico del código

1.1 Arquitectura general

El proyecto se apoya en un micro-framework propio (**gale**) que separa tres responsabilidades clásicas de un juego: **Game** (bucle principal y ventana), **StateMachine** / **BaseState** (gestión de pantallas/estados) e **InputHandler** (mapeo de teclas a acciones semánticas). **src/snake_game.py** instancia un único estado, **PlayState**, registrado bajo la clave **"play"**. No existen todavía pantallas de menú, pausa o "game over" como estados independientes: el fin de partida se resuelve *dentro* del mismo **PlayState**,

mediante una bandera (`world.game_over`) que la vista interpreta para dibujar una superposición y que el controlador interpreta para aceptar la acción `"restart"` .

Esto es relevante para el estudio: cualquier mecánica que dependa de una nueva pantalla (menú de selección, tienda, etc.) exigirá primero ampliar el `StateMachine` , mientras que las mecánicas que sólo modifican *reglas de la partida en curso* —como las tres que se proponen en este informe— pueden implementarse sin tocar la máquina de estados en absoluto, lo que reduce drásticamente la superficie de error accesible a un participante sin experiencia previa en el motor.

1.2 Bucle principal y temporización

El "tick" de juego no corre al ritmo de los cuadros por segundo, sino a un paso fijo acumulado:

`World.update(dt)` acumula `dt` en `self._move_timer` y ejecuta `self._step()` tantas veces como `move_interval` quepa en el acumulador (patrón *fixed-timestep accumulator*). Esto desacopla la velocidad de la serpiente del framerate de renderizado y es, en sí mismo, un punto de extensión limpio para una de las mecánicas propuestas (alterar la velocidad de movimiento por zona sólo exige modificar el valor que se compara contra `move_interval` , no reescribir el bucle).

El movimiento discreto vive enteramente en `src/entities/snake.py` : la serpiente es una lista de celdas `(x, y)` (`List[Tuple[int, int]]`), la cabeza es `segments[0]` , y el avance inserta una nueva cabeza y recorta la cola salvo que haya crecimiento pendiente (`grow_pending`). El sistema de colisión distingue explícitamente el *siguiente* paso (`next_head` , `would_collide_with_self`) del paso ya confirmado, lo cual permite decidir un "game over" **antes** de mutar el estado real — un detalle que cualquier mecánica nueva de colisión (p. ej. venenos, zonas letales) debe respetar para no romper la garantía de que la serpiente "nunca llega a pisar" la celda que la mata.

1.3 Representación de la grilla y convención de coordenadas

La grilla es implícita: no existe una matriz 2D materializada, sino pares `(x, y)` con `x ∈ [0, grid_cols)` , `y ∈ [0, grid_rows)` , y un enum `Direction` que expone `dx / dy` y su `opposite` . Todo el álgebra de movimiento ocurre en **espacio de celdas**, y sólo el renderer (`WorldRenderer._cell_topleft / _cell_center`) convierte celdas a píxeles multiplicando por `settings.CELL_SIZE` . Esta frontera es limpia hoy, pero es exactamente el punto donde el "vibe coding" tiende a introducir errores de unidades (celdas vs. píxeles) al añadir mecánicas espaciales nuevas —ver §3.1.

1.4 Separación modelo–vista–controlador

Capa	Archivo(s)	Responsabilidad	¿Conoce pygame?
Modelo	<code>src/world.py</code> , <code>src/entities/*.py</code> , <code>src/direction.py</code>	Reglas de juego puras: grilla, serpiente, comida, colisión, puntaje	No

Vista	<code>src/rendering/world_renderer.py</code>	Dibuja el estado del modelo; sólo lee atributos públicos de <code>World</code>	Sí
Controlador	<code>src/states/play_state.py</code>	Traduce input → acciones del modelo; dispara SFX/música en eventos	Sí
Configuración	<code>settings.py</code>	Constantes, assets, sonidos, mapeo de teclas	Sí (carga assets)

Esta separación es deliberada y consistente: `World` nunca importa `pygame`, lo que significa que toda mecánica de *reglas* (qué se puede comer, qué zona altera qué) debería vivir ahí y exponerse como atributos/propiedades públicos de solo lectura —el mismo patrón ya usado por `game_over`, `mouth_open`, `impact_cell`— para que el renderer pueda dibujar el efecto sin acoplarse a la lógica.

1.5 Idiomas reutilizables ya presentes en el código

El código ya establece convenciones que las nuevas mecánicas deberían imitar en lugar de reinventar:

- **Eventos de un solo disparo:** `_impact_event` / `_eat_event` junto a `consume_impact_event()` / `consume_eat_event()` evitan que un evento se procese en cada frame mientras la condición siga siendo verdadera. Cualquier mecánica nueva que dispare un sonido o una animación (p. ej. "entrar a una zona", "activar el filtro") debe seguir este mismo patrón.
- **Crecimiento parametrizable:** `Snake.grow(amount: int = 1)` ya acepta una cantidad variable, aunque hoy sólo se invoca con el valor por defecto. Es el enganche natural para una mecánica de comida con valor numérico.
- **Reinicio centralizado:** `World.reset()` es el único lugar que reinicializa una partida (no se reconstruye el objeto `World` entero al reiniciar). **Cualquier atributo nuevo de estado debe añadirse tanto en `__init__` / uso normal como en `reset()`**, o quedará "contaminado" entre partidas —un error extremadamente común y fácil de verificar en código generado por IA.
- **Constantes centralizadas:** todo número mágico de diseño (radios, duraciones, colores, umbrales) vive en `settings.py`, nunca hardcodeado dentro de la lógica. Es un criterio de calidad objetivo y fácil de auditar en las entregas del estudio.

Conclusión del diagnóstico: la arquitectura es modular y admite bien código externo porque (a) el modelo es puro y está desacoplado de `pygame`, (b) existe un único punto de mutación por tick (`World._step()`), (c) la vista sólo lee estado público, y (d) hay convenciones ya visibles (eventos de un solo disparo, reinicio centralizado, constantes en `settings.py`) que actúan como "barandillas" implícitas frente a una IA que, sin supervisión, tiende a esparcir lógica nueva por el archivo que tenga abierto en vez de respetar estas fronteras.

2. Recomendaciones de mecánicas (rango de filtrado)

Las tres mecánicas siguientes cubren, respectivamente, los ejes espacial, de valores y de coordenadas solicitados. Se añade una cuarta, opcional, que combina las anteriores para un nivel de dificultad avanzado dentro del mismo estudio.

2.1 Radar Olfativo

Espacial · Nivel base

La serpiente obtiene un **radio de detección** r (en celdas de grilla) centrado en su cabeza. Las manzanas fuera de r se renderizan ocultas (p. ej. como un signo de interrogación o un sprite "niebla"); en cuanto la distancia entre la cabeza y la manzana entra dentro de r , la manzana se revela con su sprite normal. Como variante de la misma idea, un obstáculo (roca, trampa) dentro del radio puede telegrafarse (parpadeo, halo) antes de resultar letal, dando al jugador una ventana de reacción proporcional a r .

Regla formal: para cada manzana a con posición (ax, ay) y cabeza en (hx, hy) , se define *revelada* $\Leftrightarrow \text{distancia}(a, \text{cabeza}) \leq r$, usando distancia Chebyshev ($\max(|ax-hx|, |ay-hy|)$) para que el radio se perciba como un cuadrado alineado a la grilla —coherente con un juego de casillas— en vez de un círculo euclidiano que no calza visualmente con los sprites cuadrados existentes.

2.2 Filtro Metabólico de Valores

Valores · Nivel intermedio

Se introducen 2–3 tipos de manzana, cada una con un **valor numérico** (p. ej. -5 , $+5$, $+15$) que combina "puntaje" y "cantidad de crecimiento". La serpiente porta un **filtro activo**: un rango $[v_{\min}, v_{\max}]$ (constante al inicio, opcionalmente ampliable con un power-up). Sólo puede *comer* con seguridad las manzanas cuyo valor cae dentro del rango; las que quedan fuera son intangibles (la serpiente las atraviesa sin efecto) o tóxicas (game over), según la variante que se elija para el estudio.

Regla formal: al pisar la celda de una manzana con valor v : si $v_{\min} \leq v \leq v_{\max} \rightarrow \text{snake.grow}(v)$, $\text{score} += v$, se elimina la manzana y se dispara el evento "comida válida"; si no, se dispara el evento "filtro rechazado" (game over, o simplemente ausencia de efecto, según diseño).

2.3 Zonas de Alteración Cartesiana

Coordenadas · Nivel intermedio

Se definen una o más regiones rectangulares del mapa mediante rangos de coordenadas de grilla, p. ej. $x \in [5, 9]$, $y \in [2, 4]$. Mientras la cabeza de la serpiente esté dentro de esos rangos, cambia una regla física del juego: velocidad de movimiento (duplicar/reducir a la mitad el `move_interval` efectivo), inversión de controles izquierda/derecha, o inversión arriba/abajo. Al salir de la zona, la física vuelve a su valor base.

Regla formal: $\text{en_zona}(hx, hy) \Leftrightarrow x_{\min} \leq hx \leq x_{\max} \wedge y_{\min} \leq hy \leq y_{\max}$. El efecto se aplica como una transformación condicional sobre el `move_interval` efectivo (velocidad) o sobre el

mapeo acción → `Direction` (controles), nunca como una mutación permanente del valor base en `settings.py`.

2.4 (Opcional, avanzado) Filtro Compuesto: Radar de Valor por Zona

Combinado · Nivel avanzado

Para participantes con mayor dominio, se puede pedir la combinación de las tres ideas: el radio de detección de 2.1 sólo revela manzanas cuyo valor cae dentro del filtro de 2.2, y ese rango `[v_min, v_max]` se desplaza o se estrecha según la zona cartesiana (2.3) en la que se encuentre la cabeza. Esto no es una mecánica nueva desde cero, sino una prueba de **composición** de las tres anteriores, útil como tarea de cierre en el estudio para medir si los participantes logran hacer que un prompt de IA reutilice código ya escrito en vez de duplicarlo.

Matriz de dificultad orientativa (para el diseño del estudio)

Mecánica	Archivos tocados	Complejidad conceptual	Riesgo típico de la IA
2.1 Radar Olfativo	3	Baja–media	Confusión de unidades (píxeles vs. celdas)
2.2 Filtro de Valores	4	Media	Desincronización comer/colisionar; olvidar <code>spawn_one</code>
2.3 Zonas Cartesianas	3–4	Media	Aplicar el efecto en el ciclo equivocado (evento vs. estado)
2.4 Filtro Compuesto	4–5 (reutiliza)	Alta	Duplicar lógica ya existente en vez de componerla

3. Viabilidad para vibe coding

Para cada mecánica se documenta (a) una instrucción de referencia razonable que un participante podría escribirle a un asistente de IA, y (b) los errores de lógica espacial, de arreglos o de sincronización de estado que son esperables en la primera respuesta de la IA —y que el participante deberá detectar leyendo el *diff*, no simplemente aceptando el resultado a ciegas. Esta lista de errores es, en sí misma, un instrumento de evaluación útil para el estudio: permite puntuar si el participante detectó el fallo, o si el juego "se ve roto" y aun así lo dio por bueno.

3.1 Radar Olfativo

“Prompt de referencia: *"Añade un radio de detección de 4 celdas alrededor de la cabeza de la serpiente. Las manzanas fuera de ese radio deben dibujarse ocultas (un ícono de interrogación); al entrar la cabeza dentro del radio, se revela la manzana con su sprite normal."*

- **Mezcla de unidades:** la IA calcula con frecuencia la distancia usando las coordenadas de *renderizado* (píxeles, ya multiplicadas por `CELL_SIZE`) contra un radio pensado en celdas, o viceversa —el radio “de 4” se vuelve invisible (demasiado pequeño en píxeles) o siempre activo (demasiado grande en celdas). El participante debe verificar explícitamente en qué espacio se hace la comparación.
- **Círculo euclidiano por defecto:** sin instrucción explícita, la IA suele usar `sqrt(dx**2+dy**2) ≤ r` (círculo real), que en una grilla de sprites cuadrados de `settings.CELL_SIZE` px se percibe "torcido" en los bordes del radio. Chebyshev/Manhattan producen un borde que calza con la cuadrícula.
- **Estado no reiniciado:** si se guarda una bandera "ya revelada" en el objeto `Apple`, y una nueva manzana se genera vía `FoodField.spawn_one()` sin inicializar ese atributo, la IA puede lanzar `AttributeError` en la primera manzana de reemplazo, o peor, dejarla "revelada" por defecto sin darse cuenta.
- **Confusión fila/columna:** como los `segments` son tuplas `(x, y)` y no `(fila, columna)`, es común que la IA transponga el orden al escribir el bucle de comparación contra `grid_rows / grid_cols`, produciendo un radio "elíptico" en mapas no cuadrados (20×15 en este proyecto).

3.2 Filtro Metabólico de Valores

“Prompt de referencia: *"Crea 3 tipos de manzana con distinto valor (-5, +5, +15). La serpiente sólo puede comer con seguridad las manzanas cuyo valor esté entre un mínimo y un máximo definidos por un filtro activo. Si come una fuera de ese rango, pierde."*

- **Comer vs. colisionar sin distinguir:** la IA a menudo seguirá tratando "pisar la celda de la manzana" y "comerla" como sinónimos, sin añadir la rama condicional que decide si el valor cae dentro del filtro.

El resultado observable es que *todas* las manzanas se comen igual, y la mecánica queda decorativa (sólo cambia el color, no el comportamiento).

- **Rango inclusivo/exclusivo ambiguo:** pedir "entre un mínimo y un máximo" no especifica si los límites cuentan; una IA puede usar `<` donde el participante esperaba `≤`, lo que hace que una manzana con valor exactamente igual al límite del filtro se comporte de forma contraria a la esperada por el jugador.
- **Olvidar `spawn_one`:** es frecuente que la IA modifique `Apple.__init__` y `apple_at()/remove()` pero no `FoodField.spawn_one()`, que es quien realmente instancia manzanas nuevas tras cada consumo —el bug sólo aparece después de comer la primera manzana, lo que lo hace fácil de pasar por alto en una prueba rápida.
- **Crecimiento no proporcional:** aunque `Snake.grow(amount)` ya admite una cantidad, una IA sin instrucción explícita seguirá llamando `snake.grow()` (equivalente a `grow(1)`) ignorando el valor de la manzana, con lo que "valor" sólo afecta el puntaje y no el tamaño —una inconsistencia de diseño que el participante debe decidir si es intencional y, si no, corregir.

3.3 Zonas de Alteración Cartesiana

“Prompt de referencia: *"Define una zona rectangular entre las columnas 5 y 9, filas 2 y 4, donde los controles izquierda/derecha se inviertan mientras la cabeza esté dentro de ese rango de coordenadas."*

- **Aplicar el efecto en el lugar equivocado del ciclo:** el punto correcto para invertir un control es donde se traduce la acción de entrada a un `Direction` (en `PlayState.on_input`, antes de llamar a `world.set_direction`), no dentro de `World.update()` / `_step()`. Una IA que lo implemente en el segundo lugar tiende a reaplicar la inversión en cada tick sobre un estado ya invertido, produciendo parpadeo de dirección en vez de una inversión estable mientras se permanece en la zona.
- **Límites de rango con errores de uno:** `x_min ≤ x ≤ x_max` frente a `x_min < x < x_max` desplaza el borde de la zona en una celda; en una grilla pequeña (20×15) esto es fácil de notar jugando, pero fácil de pasar por alto leyendo sólo el código.
- **No respetar el guard de fin de partida:** `Snake.set_direction` y `World.set_direction` ya ignoran cambios de dirección si `world.finished`. Una IA que añada la inversión de controles en un punto distinto (p. ej. directamente sobre el `input_id` en `PlayState.on_input`) puede olvidar ese guard y permitir "trampas" de dirección tras el game over.
- **Zona en píxeles en vez de celdas:** igual que en 3.1, si la IA define el rectángulo con las coordenadas de renderizado (multiplicadas por `CELL_SIZE`) pero compara contra `snake.head` (en celdas), la zona nunca se activa o se activa en una posición desplazada del mapa.

3.4 Filtro Compuesto (avanzado)

- **Duplicación en vez de composición:** el riesgo dominante en esta tarea no es un bug de lógica sino de diseño: una IA instruida a "combinar" las tres mecánicas anteriores sin apuntarle a los archivos/funciones concretos ya escritos tiende a reescribir una versión paralela de `_free_cells`,

del cálculo de distancia o del chequeo de zona, en vez de invocar el código ya existente. Es la tarea ideal para evaluar si el participante sabe pedirle a la IA que *reutilice* en lugar de *reinventar*.

4. Puntos de integración

A continuación, el mapa concreto de clases/funciones/archivos donde cada mecánica debe inyectarse, siguiendo las convenciones ya identificadas en el diagnóstico (§1.5).

4.1 Radar Olfativo

Archivo	Elemento	Cambio
<code>settings.py</code>	constantes	<code>RADAR_RANGE_CELLS</code> , color/ícono del sprite "oculto"
<code>src/entities/food.py</code>	<code>Apple</code>	nuevo atributo <code>revealed: bool</code> (o calcularlo sin estado, ver nota abajo)
<code>src/entities/food.py</code>	<code>FoodField</code>	método <code>apples_within(cell, radius)</code> reutilizable
<code>src/world.py</code>	<code>World.update()</code> / nueva propiedad	recalcular qué manzanas están "reveladas" cada tick, exponerlo como propiedad pública de solo lectura
<code>src/rendering/world_renderer.py</code>	<code>_render_food</code>	elegir sprite normal u "oculto" según el estado revelado del modelo

Nota de diseño: es preferible **no** guardar `revealed` como estado persistente en `Apple`, sino recalcularlo cada frame a partir de la distancia actual (célula pura, sin memoria) —evita por completo el bug de "estado no reiniciado" descrito en §3.1, a costa de que una manzana ya vista puede "re-ocultarse" si el jugador se aleja. Vale la pena plantear ambas variantes a los participantes y que decidan cuál prefieren, documentando el porqué.

4.2 Filtro Metabólico de Valores

Archivo	Elemento	Cambio
<code>settings.py</code>	constantes	<code>APPLE_VALUES</code> (tipos y valores), <code>FILTER_RANGE_DEFAULT</code> , colores por tipo
<code>src/entities/food.py</code>	<code>Apple.__init__</code>	nuevo atributo <code>value: int</code>

<code>src/entities/food.py</code>	<code>FoodField.spawn_one</code>	asignar tipo/valor aleatorio a la manzana generada (punto de fallo típico, ver §3.2)
<code>src/entities/snake.py</code>	<code>Snake</code>	nuevos atributos <code>filter_min</code> / <code>filter_max</code> ; reutilizar <code>grow(amount)</code> ya existente
<code>src/world.py</code>	<code>World._step()</code>	al detectar <code>eaten_apple</code> , ramificar según <code>filter_min ≤ value ≤ filter_max</code> ; en caso de rechazo, decidir "intangible" o "game over"
<code>src/world.py</code>	<code>World.reset()</code>	reinicializar <code>filter_min</code> / <code>filter_max</code> a sus valores base (crítico, ver §1.5)
<code>src/rendering/world_renderer.py</code>	<code>_render_food</code> , <code>_render_hud</code>	color por tipo de manzana; mostrar el rango de filtro activo en el HUD

4.3 Zonas de Alteración Cartesiana

Archivo	Elemento	Cambio
<code>settings.py</code>	constantes	<code>ALTERATION_ZONES</code> : lista de rectángulos (<code>x_min</code> , <code>x_max</code> , <code>y_min</code> , <code>y_max</code> , efecto)
<code>src/world.py</code>	nueva propiedad, p. ej. <code>current_zone_effect</code>	determinar en qué zona cae <code>snake.head</code> cada tick
<code>src/world.py</code>	<code>World._step()</code>	si el efecto es "velocidad", ajustar el <code>move_interval</code> efectivo usado por el acumulador
<code>src/states/play_state.py</code>	<code>on_input</code>	si el efecto activo es "invertir controles", aplicar <code>Direction.opposite</code> (ya existe en <code>src/direction.py</code> , no reimplementar) antes de llamar a <code>world.set_direction</code>

<code>src/rendering/world_renderer.py</code>	<code>render()</code>	dibujar un tinte/overlay sobre la zona activa para que el jugador la perciba antes de entrar
--	-----------------------	--

4.4 Convenciones transversales que deben respetarse en las tres mecánicas

- Todo umbral/constante de diseño se declara en `settings.py`, nunca hardcodeado en `World`, `Snake` o `PlayState`.
- Todo estado nuevo que sea persistente entre ticks debe reiniciarse explícitamente en `World.reset()`.
- Todo dato que la vista necesite dibujar (radio activo, zona actual, filtro vigente) se expone como atributo o propiedad pública de `World`, replicando el patrón de `game_over` / `mouth_open` / `impact_cell`; el renderer nunca debe importar reglas de negocio.
- Todo evento puntual (entrar a una zona, rechazo del filtro) sigue el patrón de bandera + `consume_*_event()` ya usado por `_impact_event` / `_eat_event`, para enganchar sonido en `PlayState.update()` sin re-disparar en cada frame.

5. Conclusión y nota metodológica

Las tres mecánicas propuestas comparten una misma "forma" —una función de pertenencia sobre un rango (espacial, numérico o de coordenadas) que habilita o modifica un comportamiento— pero requieren tocar puntos de integración distintos y activan clases de error distintas en el código generado por IA. Esto las hace un conjunto experimental adecuado para un estudio de vibe coding: permiten comparar, con una variable de diseño constante ("rango de filtrado"), cómo distintos participantes instruyen a la IA y qué tan bien detectan fallos de unidades, de sincronización de estado o de estructura de datos en el resultado, sin depender de que dominen previamente la arquitectura completa del proyecto.